

Lesson 8 — Authentication and Security

Node.js/Express

Game Plan

- Warm-Up
- Why Authentication Matters
- JSON Web Tokens (JWT)
- HttpOnly Cookies
- CSRF Protection
- Auth Middleware
- Other Security Packages
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

Think of a website you use that requires you to log in.

- After you log in once, how does it "remember" you across requests?
- What do you think happens under the hood?

The Two Problems We're Fixing

Currently your app has:

1. **Only one user can be logged in at a time** — `global.user_id`
2. **Anyone can access data as the logged-in user** — no real session

Both problems stem from using a global variable. This week, you fix them with a real authentication system.

How Sessions Work

When a user logs in:

1. Server **verifies** their email + password
2. Server creates a **signed token** (JWT) with their user ID
3. Token is sent back in an **HttpOnly cookie**
4. Browser sends that cookie with **every subsequent request**
5. Auth middleware **verifies** the token on protected routes

No more global — every request proves who's asking.

What Is a JWT?

A **JSON Web Token** has three parts, separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJF9i0jF9.XYZ_signature  
[header]                [payload]                [signature]
```

- **Header** — algorithm used
- **Payload** — claims (userId, expiry, etc.)
- **Signature** — proves the token wasn't tampered with

Anyone can **read** the payload. The signature makes it **unforgeable**.

Creating a JWT

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
  { userId: user.id, csrfToken: crypto.randomUUID() },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);
```

The `JWT_SECRET` must be a long, random string — store it in `.env`.

If an attacker gets your secret, they can forge any user's token.

Verifying a JWT

```
try {  
  const payload = jwt.verify(token, process.env.JWT_SECRET);  
  req.user = { id: payload.userId };  
  next();  
} catch (err) {  
  return res.status(401).json({ message: "Unauthorized." });  
}
```

`jwt.verify` throws if the token is expired, invalid, or tampered with.

Why HttpOnly Cookies?

Where do you store the JWT?

- **localStorage** — accessible by JavaScript → vulnerable to XSS
- **HttpOnly cookie** — inaccessible to JavaScript → safe from XSS

Set it on the server:

```
res.cookie("token", jwtToken, {  
  httpOnly: true,  
  sameSite: "Strict",  
  maxAge: 24 * 60 * 60 * 1000, // 1 day  
});
```

Reading the Cookie

With `cookie-parser` middleware:

```
app.use(cookieParser());
```

The token is now available as:

```
const token = req.cookies.token;
```

Without `cookie-parser`, you'd have to parse `req.headers.cookie` manually.

CSRF Attacks

Cross-Site Request Forgery: A malicious page tricks your browser into sending a request to your app — and the HttpOnly cookie goes along with it.

Attacker's page:

```
<form action="https://yourapp.com/api/tasks/5" method="POST">  
  <!-- browser auto-sends cookie, even from a different site -->  
</form>
```

The server can't tell if *you* sent it or the attacker did.

CSRF Defense: Tokens

Fix: Store a random **CSRF token** in the JWT.

- Return it in the **login response body** (not in a cookie)
- Front end stores it in localStorage
- Front end sends it as a **header** on every write request
- Middleware checks: header token matches token in the cookie

An attacker can send your cookie — but can't read your localStorage to get the CSRF token.

Auth Middleware (Full Version)

```
module.exports = (req, res, next) => {  
  const token = req.cookies.token;  
  if (!token) return res.status(401).json({ message: "Unauthorized." });  
  
  try {  
    const payload = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = { id: payload.userId };  
  
    // CSRF check for write operations  
    const method = req.method;  
    if (["POST", "PATCH", "DELETE"].includes(method)) {  
      const csrfHeader = req.get("X-CSRF-Token");  
      if (csrfHeader !== payload.csrfToken) {  
        return res.status(401).json({ message: "Invalid CSRF token." });  
      }  
    }  
    next();  
  } catch {  
    return res.status(401).json({ message: "Unauthorized." });  
  }  
};
```

Logging Off

Clear the cookie:

```
app.post("/api/users/logoff", authMiddleware, (req, res) => {  
  res.clearCookie("token");  
  res.json({ message: "Logged off." });  
});
```

The logoff route is **protected** by auth middleware — this prevents an attacker from forcing a logoff (which could enable spoofing attacks).

Other Security Packages

```
npm install helmet express-xss-sanitizer express-rate-limit
```

- **helmet** — sets security HTTP headers automatically
- **express-xss-sanitizer** — sanitizes request body/URL to disable scripts
- **express-rate-limit** — limits requests per IP (blocks bots)

Each one is just a middleware:

```
app.use(helmet());  
app.use(xss());  
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
```

We Do — Trace a Login

Walk through a full login sequence:

1. `POST /api/users/logon` — body: `{ email, password }`
2. Look up user by email in database
3. Verify password hash
4. Create JWT with `{ userId, csrfToken }`
5. Set `HttpOnly` cookie
6. Return `{ name, csrfToken }` in response body

Where does CSRF token get stored on the front end?

You Do (5 min)

What should the auth middleware return if:

1. No cookie is present?
2. The JWT is expired?
3. The JWT is valid but the CSRF token doesn't match on a POST?
4. The JWT is valid and CSRF token matches — what happens next?

Assignment Preview

Assignment 8 touches everything:

1. Update logon to create and set a JWT cookie
2. Update register to do the same
3. Rewrite auth middleware to verify JWT + CSRF
4. Update logoff to clear the cookie
5. Remove all uses of `global.user_id`
6. Add helmet, xss-sanitizer, rate-limit

The app's auth finally works correctly — multiple users, real sessions.

Wrap-Up

In chat:

1. Why is an HttpOnly cookie safer than localStorage for storing a session token?
2. What is a CSRF attack, and how does a CSRF token stop it?
3. What does `jwt.verify` do if the token has been tampered with?

Confidence Check

On a scale of 1–5:

How confident do you feel implementing real authentication this week?

Resources

- <https://jwt.io/> (visualize JWT tokens)
- <https://owasp.org/www-community/attacks/csrf>
- <https://www.npmjs.com/package/jsonwebtoken>
- Ask questions in Slack

Closing

This week:

Real authentication — JWTs, secure cookies, CSRF protection.

Next week:

Automated testing — write tests for your own code using Jest and Supertest.

See you then!